Case study: REPLIC, learning-based congestion control over UDP

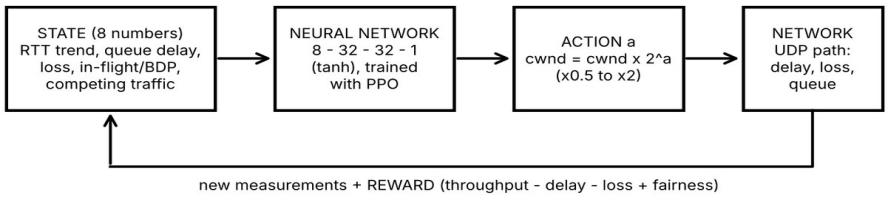
1. Application area. Reliable data transport over lossy, high-latency mobile networks (4G/5G and weak Wi-Fi) and user-space transports such as QUIC [8], where the protocol runs inside the application and can change without a kernel update. Typical uses: uploading a file, sending messages or syncing data from a phone with a poor signal.

2. Background. UDP only delivers packets: no ordering, no retransmission, no congestion control. TCP adds all three, but it lives in the kernel and cannot be modified. Its congestion control grows the sending window slowly and halves it on loss [1]. Newer algorithms such as CUBIC and BBR [4] are better, but each was hand-designed for particular network conditions.

3. Problem. The classic rule assumes that every lost packet means a full queue. On a cellular link much of the loss is radio noise, so the sender slows down although the link is free, and a short transfer takes far longer than it should. Retuning an algorithm for every new network is slow expert work. Our question: can a very small learned controller, running inside an ordinary C program, do better on such paths and still share the link fairly?

4. How REPLIC works. (a) *Transport:* a UDP protocol in C with a 13-byte header, CRC-16 to detect damage, sequence numbers, Selective Repeat with a 256-packet window, and a SACK bitmap so the receiver can say exactly which packets arrived. The sender retransmits when three later packets have been acknowledged or a Jacobson timer (with Karn's rule) expires, and an optional XOR parity packet per eight data packets repairs one loss without any retransmission. (b) *Controller:* every control interval the sender measures eight numbers (Fig 4) and feeds them to a small neural network. Its output a, between -1 and 1, scales the window by 2 to the power a, so one step can halve or double it. (c) *Training:* the network is trained in simulation with Proximal Policy Optimization [7]. The reward favours throughput and penalises delay and loss, with a Jain-fairness term so that the flow does not take more than its share. The trained weights are compiled into the C file, so no machine-learning library is needed at run time.

Fig 4. The learning loop



5. Results. On 30 simulated cellular-like paths a 1 MB transfer took 6.1 s with REPLIC, against 14.9 s for the original fixed-window AIMD sender, 12.9 s for tuned AIMD, 12.5 s for user-space CUBIC and 6.6 s for a BBR-style controller. With forward error correction REPLIC needed 5.2 s. Paired on the same paths, REPLIC takes 0.41 of the time of the original AIMD sender and is faster on all 30 paths. It also stays fair: against CUBIC competitors the Jain index is 0.974 with one competitor and 0.958 with two, better than the earlier static agent (0.942, 0.881). A network trained without the fairness term took up to 1.72x its fair share, which shows the reward term is doing real work.

Fig 5. Transfer time, 30 paths

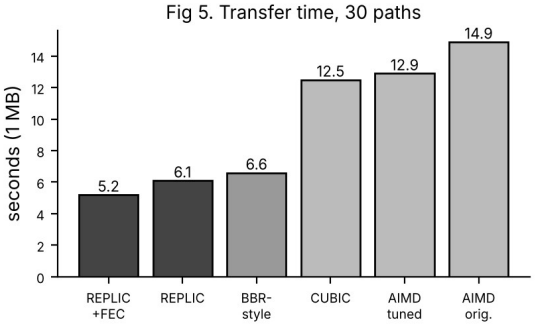
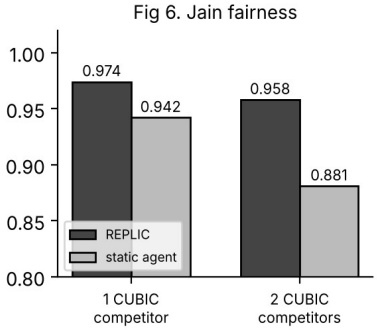


Fig 6. Jain fairness



6. Limits (honest reporting). Kernel TCP on Linux netem was still faster than REPLIC in every profile we tried (about 1.3 to 1.9x against kernel CUBIC); REPLIC with FEC ties kernel CUBIC only on cellular-like paths. The other baselines are our own user-space versions, so the comparison is fair among them but is not a claim over the kernel.

7. Future work. Close the gap to the kernel with a faster send path, and train the controller on traces measured on real networks.

8. Conclusion. A controller of about 1,300 multiplications per decision, trained by reinforcement learning and written in plain C, beats fixed-rule controllers on lossy long-delay paths and stays fair. A new kind of network needs retraining, not a redesign.

9. Researcher in the area. Keith Winstein (Stanford University), co-author of Remy [2] and the Pantheon testbed [5]. His work showed that congestion control can be generated by optimisation and compared fairly on shared testbeds, which is the approach REPLIC follows with PPO. Other related work: Aurora [3] and Orca [6].

Bibliography

[1] V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM, 1988.

[2] K. Winstein, H. Balakrishnan, "TCP ex Machina: Computer-Generated Congestion Control," ACM SIGCOMM, 2013.

[3] N. Jay et al., "A Deep Reinforcement Learning Perspective on Internet Congestion Control," ICML, 2019.

[4] N. Cardwell et al., "BBR: Congestion-Based Congestion Control," ACM Queue 14(5), 2016.

[5] F. Y. Yan et al., "Pantheon: the training ground for Internet congestion-control research," USENIX ATC, 2018.

[6] S. Abbasloo, C.-Y. Yen, H. J. Chao, "Classic Meets Modern: A Pragmatic Learning-Based Congestion Control for the Internet," ACM SIGCOMM, 2020.

[7] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[8] A. Langley et al., "The QUIC Transport Protocol: Design and Internet-Scale Deployment," ACM SIGCOMM, 2017.